Array Operations in C

1. Introduction

Arrays allow us to store **multiple values of the same data type** in a single variable.

After creating an array, we often need to **perform operations on the elements** stored in it. These are called **array operations**.

Common array operations include:

1. Traversing an array
2. Inserting elements
3. Deleting elements
4. Searching for elements
5. Updating elements
6. Finding maximum and minimum values
7. Calculating the sum and average

1. Traversing an Array

Traversing means **visiting each element of the array one by one** to process or display it.

This is usually done using a **loop**.

Example: Display Array Elements

```
#include <stdio.h>
int main()
{
   int numbers[5] = {10, 20, 30, 40, 50};

   for(int i = 0; i < 5; i++)
   {
      printf("%d\n", numbers[i]);
   }

   return 0;
}
```

Output

10

20

30

40

50

Explanation:
The loop starts at **index 0** and continues until **index 4**.

2. Inserting an Element into an Array

Insertion means **adding a new element at a specific position** in the array.

When inserting an element, other elements may need to **shift to the right**.

Example

Initial array:

10 20 30 40

Insert **25 at position 2**

Result:

10 25 20 30 40

Example Program

```
#include <stdio.h>

int main()
{
   int arr[6] = {10,20,30,40};
   int pos = 2;
   int value = 25;
   int size = 4;

   for(int i = size; i >= pos; i--)
```

```
    {
        arr[i] = arr[i-1];
    }

    arr[pos-1] = value;

    size++;

    for(int i = 0; i < size; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

3. Deleting an Element from an Array

Deletion removes an element and **shifts remaining elements to the left**.

Example:

Original array

10 20 30 40 50

Delete element at position **3**

Result:

10 20 40 50

Example Program

```
#include <stdio.h>

int main()
{
    int arr[5] = {10,20,30,40,50};
    int pos = 3;
    int size = 5;
```

```
    for(int i = pos-1; i < size-1; i++)
    {
        arr[i] = arr[i+1];
    }

    size--;

    for(int i = 0; i < size; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

4. Searching an Element

Searching means **finding whether a specific value exists in the array**.

The simplest method is **Linear Search**.

Example

Search for **30** in:

10 20 30 40 50

Program

```
#include <stdio.h>

int main()
{
   int arr[5] = {10,20,30,40,50};
   int key = 30;
   int found = 0;

   for(int i = 0; i < 5; i++)
   {
      if(arr[i] == key)
      {
         found = 1;
         printf("Element found at position %d", i+1);
         break;
      }
   }

   if(found == 0)
   {
      printf("Element not found");
   }

   return 0;
}
```

5. Updating an Element

Updating means **changing the value of an existing element**.

Example:

Original: 10 20 30 40

Update index 2 to 50

Result: 10 20 50 40

Example

```
arr[2] = 50;
```

6. Finding the Maximum Value

Example array:

12 45 7 89 34

Maximum = **89**

Program

```
#include <stdio.h>

int main()
{
   int arr[5] = {12,45,7,89,34};
   int max = arr[0];

   for(int i = 1; i < 5; i++)
   {
      if(arr[i] > max)
      {
         max = arr[i];
      }
   }

   printf("Maximum = %d", max);

   return 0;
}
```

7. Finding the Minimum Value

Example array:

12 45 7 89 34

Minimum = **7**

Program

```
int min = arr[0];

for(int i = 1; i < 5; i++)
{
   if(arr[i] < min)
   {
      min = arr[i];
   }
}
```

8. Calculating Sum and Average

Example array:

10 20 30 40 50

Sum = **150**
Average = **30**

Program

```
#include <stdio.h>

int main()
{
   int arr[5] = {10,20,30,40,50};
   int sum = 0;

   for(int i = 0; i < 5; i++)
   {
      sum = sum + arr[i];
   }

   float avg = (float)sum / 5;

   printf("Sum = %d\n", sum);
   printf("Average = %.2f", avg);

   return 0;
}
```